Module 1
Make Your First Request with the Gemini API

- Welcome to the Google Gemini for developers specialization
  Video • 2 min
- Introduction to getting started with the Gemini API
  Video • 2 min
- Power up your applications with AI
  Video • 2 min
- Gemini API fundamentals
  Reading • 10 min
- Your first Gemini API call
  Video • 5 min
- Build an AI text generator
  Practice Assignment • Grade: 100%
- Exploring possibilities with the Gemini API
  Dialogue • 15 min
- Knowledge check: Gemini fundamentals
  Practice Assignment • Grade: 100%

Module 2
Model Selection and Parameters

- Choose the right model for you
  Video • 2 min
- **Model choice and model thinking**
  Reading • 10 min
- Implementing model selection and thinking
  Video • 4 min
- AI text generator
  Practice Assignment • 30 min
- Model strategy and thinking analysis
  Dialogue • 15 min
- Knowledge check: Making Gemini work for you
  Practice Assignment • 15 min

Module 3
Built-in Tools and Structured Output

Model choice and model thinking

0:03 / 11:06 1x

Choosing the right AI model and understanding its process

Selecting the appropriate tool is a foundational first step in any project. Google's Gemini API offers developers a powerful set of tools, primarily the Gemini Pro and Gemini Flash models. Each model is designed for different kinds of tasks, and making a strategic choice between them is a fundamental skill.

Sometimes, building with AI can feel like working with a "black box": a developer sends in an input and gets an output, without much visibility into what happened in between. Thinking-capable models help change this by significantly strengthening the model's reasoning and multi-step planning abilities, making them especially effective for complex tasks like coding, advanced mathematics, and data analysis. They're also particularly powerful for agentic workflows, where you want the model to carry out a sequence of different actions or tool calls: thinking allows the model to first sketch out a coherent plan of the steps it needs to take before it starts executing. This upfront planning improves reliability, reduces unnecessary calls, and makes it easier to orchestrate complex tasks end to end. While the thinking trace summary is not a totally reliable or infallible record of the model's internal process, it can still provide useful insight into how the model approached a problem. Those insights, combined with choosing the right model for the task, help developers debug issues, refine prompts, and progress from simple API calls to sophisticated, efficient, and reliable AI systems.

Choosing the right tool: Gemini Pro vs. Gemini Flash

The decision between using Gemini Pro or Gemini Flash depends on balancing three key factors: the quality of the response, the speed at which it is delivered (latency), and the cost of the operation. These factors often cause tension with one another. A higher-quality response may take longer and cost more, while a faster, cheaper response may not include the same level of contextual depth or reasoning detail.

Developers should begin by defining the goals of their application—whether precision, scale, or responsiveness—and what matters most. A model used for analytical reporting or multi-step planning will benefit from deeper reasoning, while one used for real-time interaction must prioritize response speed and cost efficiency. It's also helpful to consider operational constraints such as user concurrency, expected query volume, and the tolerance for small variations in output quality.

Gemini Pro

Gemini Pro is the specialist model, designed for tasks that require deep understanding, complex reasoning, and nuanced output. It excels at tasks that involve multiple steps of logic, creative writing, or in-depth analysis of complex information. Because it performs more computation to generate its answers, Gemini Pro has a higher cost per token. A token is a piece of a word; for example, the word "chatbot" might be one or two tokens. You are charged based on the number of tokens in your input and the number of tokens in the model's output.

Consider a fictitious financial technology company, "FinAnalytica." Their main product is a service that generates detailed market analysis reports for investment advisors. An advisor provides raw data, including stock performance charts, news articles, and company financial statements. The application needs to analyze this mixed information, identify key trends, and produce a well-structured, 10-page report. This is a perfect use case for Gemini Pro. The task requires a high degree of accuracy and the ability to follow complex instructions. The extra cost and slightly longer response time are acceptable trade-offs for the high quality and reliability that FinAnalytica's clients demand.

Gemini Flash

On the other hand, Gemini Flash is built for speed and efficiency. It is the ideal choice for applications that need to handle a large volume of requests quickly and at a lower cost. Gemini Flash Lite is a closely related variant that pushes this even further, offering the fastest, most cost-efficient option in the Flash family for ultra-high-throughput workloads. Compared with standard Flash, Flash Lite trades off some depth and versatility for even lower latency and price, making it a strong fit for very simple, repetitive tasks like lightweight tagging, routing, or basic field extraction at massive scale. It is perfect for applications like real-time data extraction, large-scale summarization, or other tasks where a near-instant response is more important than deep, creative analysis.

To illustrate, consider "AccuBill AI," a fictitious company providing automated invoice processing for businesses. Their service needs to extract key information—such as vendor name, invoice date, and total amount due—from thousands of uploaded documents per hour. For AccuBill AI, the critical factors are processing speed and cost per document. The task is relatively straightforward for each invoice, but the sheer volume requires high throughput. Gemini Flash is the perfect model for this use case. It can perform the high-speed information extraction required at a cost that makes the service scalable and profitable.

Making the optimal choice

The choice is not always so clear. Sometimes, a task might seem complex enough to require Gemini Pro, but a well-designed prompt can allow Gemini Flash to perform it successfully. For example, a team building a tool to extract key information from business documents might initially choose Gemini Pro for its perceived accuracy. However, by providing very clear instructions and examples in their prompt, they might discover that Gemini Flash can perform the extraction with 99% accuracy at a much lower cost and with less latency. The following table compares Gemini Pro and Gemini Flash.

Feature	Gemini Pro	Gemini Flash
Primary use case	Complex analysis, multi-step reasoning, longer or more nuanced tasks	High-volume, low-latency workloads; real-time/agentic use; cost-sensitive apps
Reasoning complexity	Higher—best when deep reasoning quality matters most	Good—optimized for speed and throughput with solid reasoning
Typical latency	Moderate to higher	Low
Relative cost	Higher per-token	Lower per-token

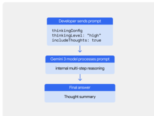
Understanding the process

Even with the right model selected, an AI's output can sometimes be surprising or incorrect. When a model produces an unexpected result, a developer needs to figure out what went wrong. This is the problem that model thinking solves. Model thinking is a feature of the Gemini API that provides a summary of the model's internal reasoning steps as it generated a response.

This transparency is incredibly valuable for debugging. If a model gives a wrong answer, the thought summary can reveal where its logic went astray. It might show that the model misinterpreted a part of the prompt, focused on the wrong keywords, or made an incorrect assumption. With this insight, a developer can refine the prompt to be clearer and more specific, guiding the model toward the correct output.

To use this feature with Gemini 3, a developer configures the model's thinking behavior rather than turning it on explicitly—thinking is enabled by default for Gemini 3 Pro. You control how much reasoning the model applies by setting thinkingLevel in the thinkingConfig, typically to "low" for faster, lighter reasoning or "high" when you want the model to spend more effort planning and solving a complex task. For development, debugging, or agentic workflows that require multi-step plans, a higher thinking level can improve solution quality, while latency-sensitive or high-volume production paths may prefer a lower level to keep costs and response times in check.

The model thinking process



A practical example of model thinking

Let's explore a practical business scenario. A developer is building a natural language-to-SQL tool for a data analytics platform. A business analyst uses the tool with the prompt: "Show me our top 5 products by sales last quarter." The application generates an SQL query that ranks products by the number of units sold, not by total revenue, which is what "sales" typically means in this context.

With model thinking enabled, the API returns a thought summary along with the incorrect SQL query. A fictitious thought summary might look like this:

1. **Identify user's goal:** The user wants a ranked list of the "top 5 products."
2. **Identify key entities and constraints:** The entities are "products" and "sales." The time constraint is "last quarter."
3. **Interpret ambiguous term:** The term "sales" can be interpreted as "number of units sold" or "total revenue."
4. **Formulate strategy:** Assume "top products by sales" means the products sold most frequently. The strategy will be to count the number of sales transactions for each product.
5. **Constraint check:** The query will filter for the correct quarter and will rank the top 5 products based on the chosen interpretation.
6. **Action:** Generate an SQL query using COUNT(order_id) grouped by product_id.

This summary immediately reveals the problem: in step 4, the model made an incorrect assumption about the business metric for "sales." Armed with this knowledge, the developer can refine the prompt to be more specific: "Show me our top 5 products by total revenue (price multiplied by quantity) last quarter." This revised prompt removes the ambiguity and guides the model to generate the correct SQL query.

Key takeaways

- **Balance trade-offs:** The choice between Gemini Pro and Gemini Flash is a balance of response quality, speed, and cost.
- **Use the right tool:** Gemini Pro is best for complex, high-quality reasoning, while Gemini Flash is optimized for high-volume, high-speed tasks.
- **Gain transparency:** Model thinking provides a look into the AI's reasoning process, which is invaluable for debugging and prompt refinement.
- **Build reliable systems:** Combining strategic model selection with the insights from model thinking allows developers to build more powerful, predictable, and cost-effective AI applications.

Mark as completed

Like Dislike Report an issue